

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MULTIDISCIPLINARY PROJECT (CO3111)

Multidisciplinary Project - Semester 232

YoloHome: SMART CLASSROOM

– Final Report –

Advisors:	Dr. Vo Thi Ngoc Chau	
Students:	Le Tran Nguyen Khoa	2152674
	Ly Minh Quang	2153722
	Dinh Viet Thanh	2152966
	Nguyen Quoc Minh Thu	2052736
	Dang Bao Tin	2152313

HO CHI MINH CITY, NOVEMBER 2024



Member list & Workload

No.	Full name	Student ID	Task	Contribution
1	Le Tran Nguyen Khoa	2152674	Support in backend development, in charge of frontend development	100%
2	Ly Minh Quang	2153722	Device connection, implementing IoT integration code and testing	100%
3	Dinh Viet Thanh	2152966	Backend development and writing report	100%
4	Nguyen Quoc Minh Thu	2052736	Frontend & backend development, collecting tasks	100%
5	Dang Bao Tin	2152313	Support in frontend development and in charge of backend development	100%



Contents

1	Requirement Elicitation	5
1.1	Project Identification	5
1.1.1	Domain context	5
1.1.2	Stakeholders and their needs	5
1.2	User Requirements	6
1.2.1	Functional requirements	6
1.2.2	Non-functional requirements	7
1.2.3	Data requirements	7
1.3	Use-Case Diagram for the whole system	8
1.3.1	Administrator Use-case	8
1.3.2	Teacher and Room Manager Use-case	15
1.3.3	Student Use-case	18
2	Hardware kit	20
3	Implementation	21
3.1	Database design	21
3.1.1	Database selection	21
3.1.2	Physical design	21
3.2	Deployment diagram	24
3.3	IoT Integration Code	24
3.3.1	Initial setup	24
3.3.2	Registration function	25
3.3.3	Events	26
3.3.4	Full IoT system	28
3.4	Finished Product	29
3.4.1	Source control	29
3.4.1.1	Distributed revision control system	29
3.4.1.2	Source code structure	29
3.4.2	Final product	32
3.4.2.1	Portal	33
3.4.2.2	Data Dashboard	33
3.4.2.3	Administrator's User Dashboard	35
3.4.2.4	IoT Dashboard	36
3.5	Demonstration	39
4	Project Evaluation	39



4.1	Summary of the project's results	39
4.2	Achievements	40
4.3	Challenges	41
4.4	Further Improvement	41

1 Requirement Elicitation

1.1 Project Identification

1.1.1 Domain context

CORE DOMAIN CONTEXT OF THIS PROJECT: Smart Classroom IoT system is an innovative project designed to integrate AI and IoT technology to create smart educational environments. As classrooms evolve, the demand for interactive and adaptive learning spaces increases. YoloHome aims to meet this demand by providing an intelligent system for environmental monitoring, secure access, and efficient automation of classroom devices.

Due to these insights, the core domain of our IoT system is centered around the automation and intelligent control of classroom devices, first with just lights and small fans, then we want to expand the system with TVs, presenters, air conditioners, projectors and computers in the classroom if possible. This system enables automated adjustments based on various factors, such as room occupancy or scheduled classes, enhancing the learning experience and operational efficiency.

SUPPORTING DOMAINS:

- Data Analytics: Leveraging AI to analyze data collected from various sensors and devices to make informed decisions and predictions.
- Device Communication: Ensuring seamless communication between different IoT devices, enabling them to operate in a coordinated manner.
- User Interaction: Developing a user-friendly interface for educators and students to interact with the system, customize settings, and receive relevant information.
- Security and Privacy: Implementing robust security measures to protect the system from unauthorized access and ensure the privacy of the users.

Through these domains, YoloHome aspires to transform traditional classrooms into smart learning environments, where technology aids in creating a more interactive, comfortable, and efficient educational experience.

1.1.2 Stakeholders and their needs

- **For Administrators:** Enhanced system management capabilities, improved security oversight, and the ability to customize the user experience.
 - *User Story:* “As an administrator, I want to easily manage user privileges and configure system settings in YoloHome, ensuring a secure and efficient environment for all users”.
- **For Teachers and Room Managers:** Simplified control over classroom technology, en-

abling more engaging and interactive lessons.

- *User Story*: “As a teacher, I want to use YoloHome to seamlessly control classroom equipment, teaching with the support of the device with ease”.
- **For Students:** Hands-on experience with smart technology, fostering practical learning and innovation.
 - *User Story*: “As a student, I want to interact with and control smart devices using YoloHome”.

1.2 User Requirements

1.2.1 Functional requirements

Priority	Category	Requirement statement	Must/Want
1	Login/Logout	The system supports login/logout for users (admin/student/teacher) via a user interface.	Must
2	IoT Dashboard	- The system allows users (admin/teacher/student) to view raw responses, outputs, and status of IoT sensors/devices via IoT Dashboard. - The system allows users (admin/teacher/student) to (1) turn on/off, (2) adjust device’s parameters sensors/devices via IoT Dashboard.	Must
3	Data Dashboard	- The system supports different representation modes for system’s data. - The system supports data exporting in at least two different formats (.pdf, .xlsx, .csv, .json, .xml).	Must
4	Threshold Trigger	- The system supports a mechanism to trigger an action on the IoT sensors/devices once received data exceeds a predefined threshold.	Must
5	AI Integration for Automation	The system supports AI-assisted to smartly adjust brightness, temperature, and device on/off functionalities.	Want
6	AI Authentication and Authorization	The system supports AI-assisted services for authentication, recognition, and permission granting for teachers and students.	Want

1.2.2 Non-functional requirements

ID	Category	Requirement statement
NF01	Availability	The system and IoT devices shall be able to operate reliably 24/7 with minimal scheduled downtime.
NF02	Reliability	The system shall enable complete manual interference of human over the IoT devices and actuators upon system failure.
NF03	Portability	IoT Gateway shall be implemented on a portable embedded computer and located within the classroom.
NF04	Scalability	The system should be designed to scale up with multiple different IoT devices and different services integrated later on.

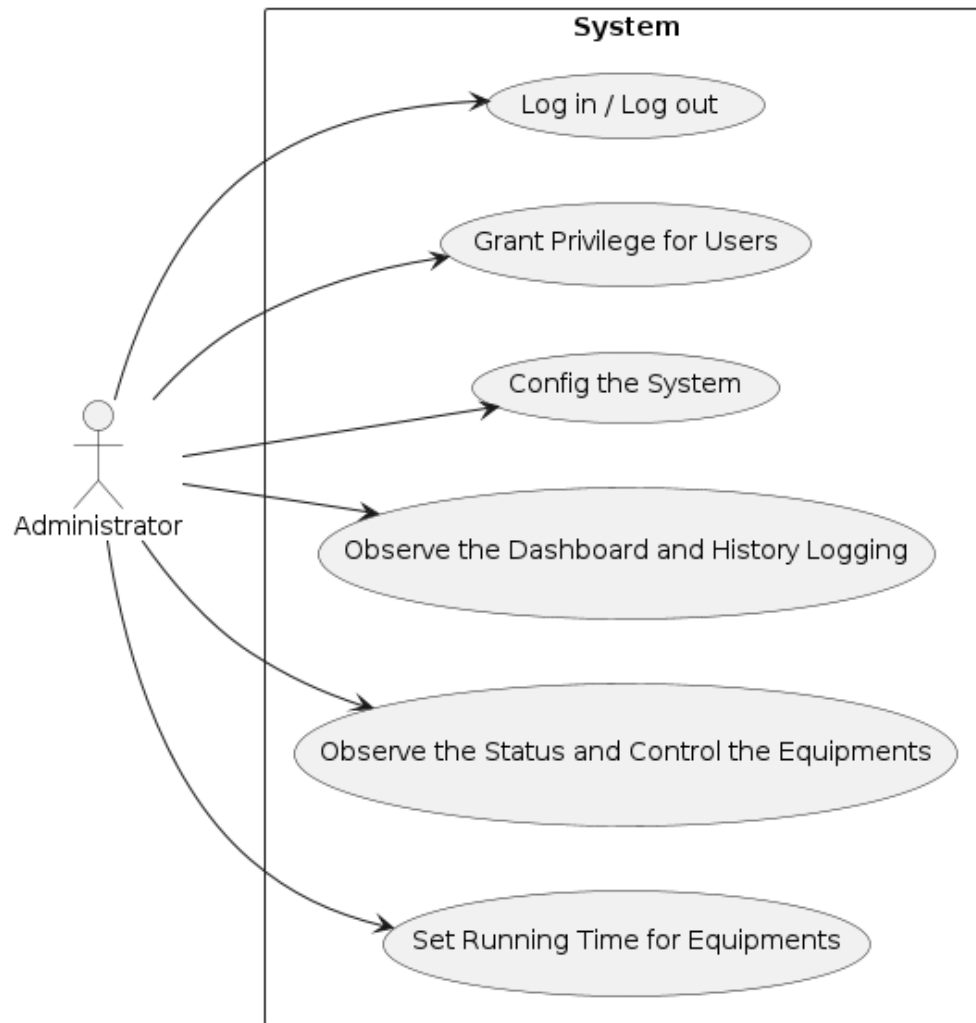
1.2.3 Data requirements

1. The system must support persistent storage for devices and sensors product's information.
2. The system must support persistent storage for system logs including trace logs, device/sensor logs, and authentication logs.
3. The system must support different levels of authorization upon its data.
4. The system allows accesses to its data via REST APIs.
5. Data streams from IoT devices to the cloud must use MQTT protocol.
6. System's data must be encrypted or hashed whether in exchange or stored in a persistent database.

1.3 Use-Case Diagram for the whole system

Requirement: Include a conceptual use-case diagram for the whole system.

1.3.1 Administrator Use-case



Log in / Log out

Use Case Name	Log in / Log out
Description	Administrators access the system by logging in and log out when they finish their session.
Actor(s)	Administrator
Trigger	Administrator wishes to access (log in) or leave (log out) the system.
Precondition(s)	For login, Administrator must have valid credentials. For logout, Administrator must be logged in.
Postcondition(s)	For login, Administrator gains access to the system functionalities. For logout, Administrator's session is securely closed.
Main flow	For Login: 1. Administrator navigates to the login page. 2. Administrator enters credentials. 3. System validates the credentials. 4. System grants access to the dashboard. For Logout: 1. Administrator selects the logout option. 2. System ends the session. 3. System displays the confirmation message.
Alternative flow	For Login: 3a. If credentials are invalid, system displays an error message. For Logout: 1a. System auto-logs out after a period of inactivity.
Exception flow	For Login: 3b. System is down. <i>Display a maintenance message.</i> For Logout: 2a. System fails to end the session properly. <i>Notify the administrator to manually check the logout process.</i>



Grant Privilege for Users

Use Case Name	Grant Privilege for Users
Description	Administrators assign or modify user roles and permissions within the system.
Actor(s)	Administrator
Trigger	Administrator decides to change the access level of a user.
Precondition(s)	Administrator is logged into the system. The user account exists in the system.
Postcondition(s)	The user's privileges are updated in the system.
Main flow	1. Administrator accesses the user management interface. 2. Administrator selects a user to modify. 3. Administrator adjusts the user's roles and permissions. 4. The system updates the user account with the new privileges. 5. The system confirms the update to the administrator.
Alternative flow	3a. Administrator cancels the operation without saving changes.
Exception flow	2a. System cannot find the selected user. <i>Notify the administrator and suggest checking the user's details.</i> 3a. System fails to update the user's privileges. <i>Notify the administrator and suggest to retry.</i>

Config the System: Enable and Disable the Equipments, Using Time

Use Case Name	Config the System
Description	Administrators configure the system settings, including enabling or disabling equipment and setting their operational schedules.
Actor(s)	Administrator
Trigger	Need to adjust the equipment settings for optimal use.
Precondition(s)	Administrator is logged in. Administrator has access to equipment settings.
Postcondition(s)	Equipment settings are updated according to the administrator's configurations.
Main flow	1. Administrator accesses the system configuration interface. 2. Administrator selects the equipment to configure. 3. Administrator enables/disables the equipment or sets the operating times. 4. System updates the equipment settings. 5. System confirms the changes to the administrator.
Alternative flow	3a. Administrator decides to leave the settings unchanged after review.
Exception flow	2a. System cannot find the selected equipment. <i>Notify the administrator and suggest verification.</i> 4a. System fails to save the settings. <i>Notify the administrator and suggest to retry.</i>



Observe the Dashboard and History Logging

Use Case Name	Observe the Dashboard and History Logging
hline Description	Administrators access and review the dashboard for real-time data and historical logs of system activities.
Actor(s)	Administrator
Trigger	Administrator navigates to the dashboard section in the system.
Precondition(s)	Administrator is logged into the system.
Postcondition(s)	Administrator gains insights from the system's operational history and current status.
Main flow	1. Administrator logs into the system. 2. Administrator navigates to the dashboard/history section. 3. The system displays real-time data and historical logs. 4. Administrator reviews the displayed information for insights and decision-making.
Alternative flow	None
Exception flow	2a. System fails to load the dashboard/history section. <i>System displays an error message and suggests to try again later.</i>

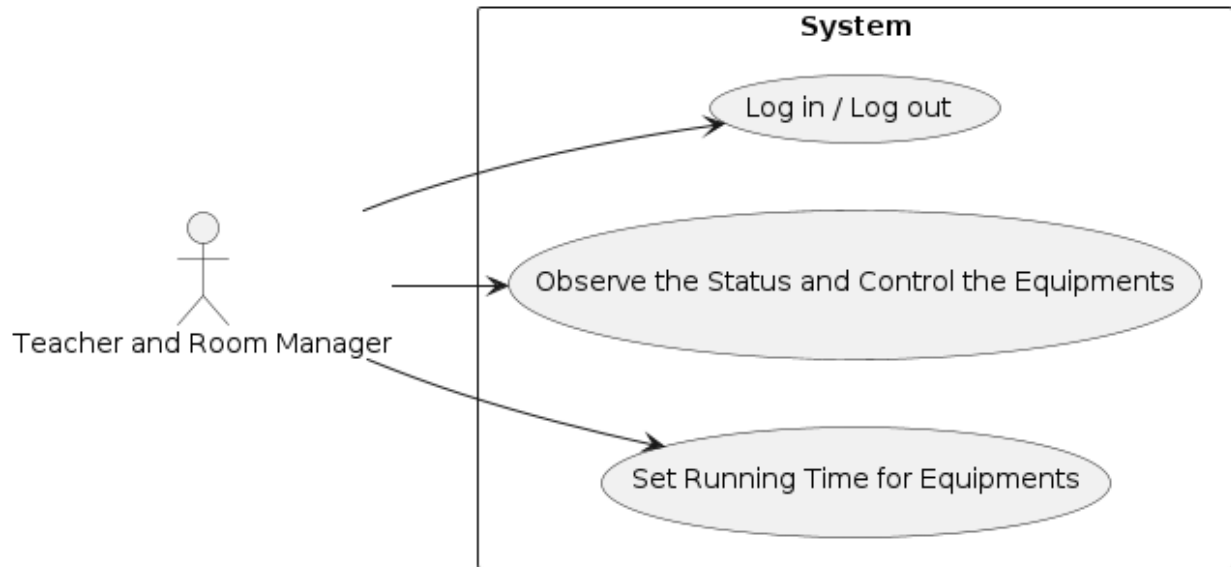
Observe the Status and Control the Equipments

Use Case Name	Observe the Status and Control the Equipments
Description	Administrators monitor the current status of equipments and remotely control them from the system interface.
Actor(s)	Administrator
Trigger	Administrator selects an equipment to view its status or to control.
Precondition(s)	Administrator is logged into the system. Equipment is correctly installed and connected to the system.
Postcondition(s)	Equipment's state is updated based on the administrator's control inputs.
Main flow	1. Administrator accesses the equipment status interface. 2. Administrator selects an equipment to observe its current status. 3. If desired, administrator sends control commands to adjust the equipment's settings. 4. The system sends the command to the equipment. 5. The equipment executes the command and its status is updated in the system.
Alternative flow	3a. Administrator decides not to make any changes after reviewing the equipment's status.
Exception flow	2a. System fails to show the current status of the selected equipment. <i>Notify the administrator and suggest troubleshooting steps.</i> 4a. System fails to send the command to the equipment. <i>Notify the administrator and suggest to retry.</i>

Set Running Time for Equipments

Use Case Name	Set Running Time for Equipments
Description	Administrators schedule the operational times for various equipments, defining when they should turn on or off.
Actor(s)	Administrator
Trigger	Administrator decides to adjust the operating schedule of an equipment.
Precondition(s)	Administrator is logged into the system. Equipment is correctly installed and connected to the system.
Postcondition(s)	Equipment's running schedule is updated in the system.
Main flow	1. Administrator accesses the equipment scheduling interface. 2. Administrator selects an equipment to schedule. 3. Administrator sets the desired running times for the selected equipment. 4. The system updates the equipment's schedule according to the administrator's settings. 5. The system confirms the update to the administrator.
Alternative flow	3a. Administrator cancels the operation without saving the changes.
Exception flow	2a. System fails to find the selected equipment. <i>Notify the administrator and suggest verifying the equipment's connectivity or installation.</i> 4a. System fails to save the updated schedule for the equipment. <i>Notify the administrator and suggest to retry.</i>

1.3.2 Teacher and Room Manager Use-case



Log in / Log out

Use Case Name	Log in / Log out
Description	Teachers and Room Managers authenticate themselves to gain access to the system for managing and controlling equipment.
Actor(s)	Teacher and Room Manager
Trigger	Teacher/Room Manager wants to access the system.
Precondition(s)	Teacher/Room Manager has valid credentials.
Postcondition(s)	Teacher/Room Manager is granted access to the system functionalities or is logged out.
Main flow	1. Teacher/Room Manager navigates to the login page. 2. Teacher/Room Manager enters their credentials. 3. The system validates the credentials. 4. Teacher/Room Manager gains access to the system (on successful login). 5. Teacher/Room Manager uses the system and logs out when done.
Alternative flow	3a. If credentials are invalid, the system displays an error message and allows the Teacher/Room Manager to try again.
Exception flow	None

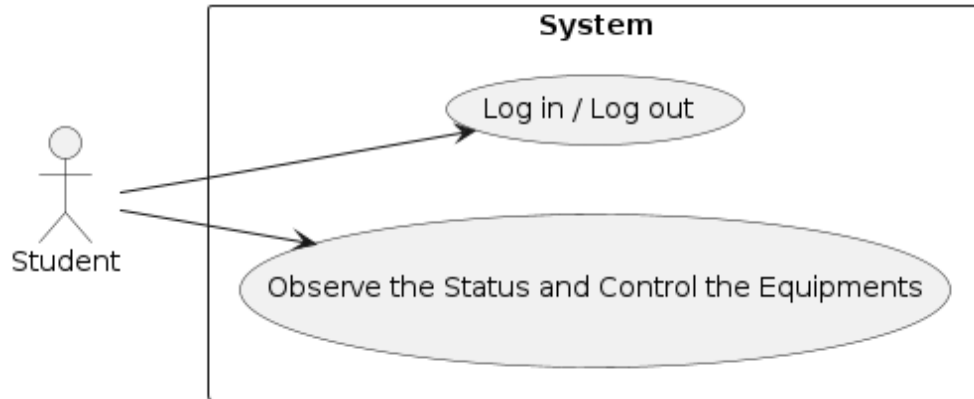
Observe the Status and Control the Equipments

Use Case Name	Observe the Status and Control the Equipments
Description	Teachers and Room Managers monitor the current status of equipment and remotely control them from the system interface.
Actor(s)	Teacher and Room Manager
Trigger	Teacher/Room Manager selects an equipment to view its status or to control.
Precondition(s)	Teacher/Room Manager is logged into the system. Equipment is correctly installed and connected to the system.
Postcondition(s)	Equipment's state is updated based on the Teacher/Room Manager's control inputs.
Main flow	1. Teacher/Room Manager accesses the equipment status interface. 2. Teacher/Room Manager selects an equipment to observe its current status. 3. If desired, Teacher/Room Manager sends control commands to adjust the equipment's settings. 4. The system sends the command to the equipment. 5. The equipment executes the command and its status is updated in the system.
Alternative flow	3a. Teacher/Room Manager decides not to make any changes after reviewing the equipment's status.
Exception flow	2a. System fails to show the current status of the selected equipment. <i>Notify the Teacher/Room Manager and suggest troubleshooting steps.</i> 4a. System fails to send the command to the equipment. <i>Notify the Teacher/Room Manager and suggest to retry.</i>

Set Running Time for Equipments

Use Case Name	Set Running Time for Equipments
Description	Teachers and Room Managers schedule the operational times for various equipments, defining when they should turn on or off.
Actor(s)	Teacher and Room Manager
Trigger	Teacher/Room Manager decides to adjust the operating schedule of an equipment.
Precondition(s)	Teacher/Room Manager is logged into the system. Equipment is correctly installed and connected to the system.
Postcondition(s)	Equipment's running schedule is updated in the system.
Main flow	1. Teacher/Room Manager accesses the equipment scheduling interface. 2. Teacher/Room Manager selects an equipment to schedule. 3. Teacher/Room Manager sets the desired running times for the selected equipment. 4. The system updates the equipment's schedule according to the Teacher/Room Manager's settings . 5. The system confirms the update to the Teacher/Room Manager.
Alternative flow	3a. Teacher/Room Manager cancels the operation without saving the changes.
Exception flow	2a. System fails to find the selected equipment. <i>Notify the Teacher/Room Manager and suggest verifying the equipment's connectivity or installation.</i> 4a. System fails to save the updated schedule for the equipment. <i>Notify the Teacher/Room Manager and suggest to retry.</i>

1.3.3 Student Use-case



Log in / Log out

Use Case Name	Log in / Log out
Description	Students authenticate themselves to gain access to the system to monitor and control equipment.
Actor(s)	Student
Trigger	Student wants to access the system.
Precondition(s)	Student has valid credentials.
Postcondition(s)	Student is granted access to the system functionalities or is logged out.
Main flow	1. Student navigates to the login page. 2. Student enters their credentials. 3. The system validates the credentials. 4. Student gains access to the system (on successful login). 5. Student uses the system and logs out when done.
Alternative flow	3a. If credentials are invalid, the system displays an error message and allows the Student to try again.
Exception flow	None

Observe the Status and Control the Equipments

Use Case Name	Observe the Status and Control the Equipments
Description	Students monitor the current status of equipment and may have the capability to control them, depending on their access rights within the system.
Actor(s)	Student
Trigger	Student selects an equipment to view its status.
Precondition(s)	Student is logged into the system. Equipment is correctly installed and connected to the system. Student has the required permissions to observe or control the equipment.
Postcondition(s)	Student gains insights into the equipment's current state and, if permitted, can control the equipment.
Main flow	1. Student accesses the equipment status interface. 2. Student selects an equipment to observe its current status. 3. If allowed, Student sends control commands to adjust the equipment's settings. 4. The system sends the command to the equipment (if the student has control permissions). 5. The equipment executes the command and its status is updated in the system (if the student has control permissions).
Alternative flow	3a. Student reviews the status but does not make any changes (if only observation is allowed).
Exception flow	2a. System fails to show the current status of the selected equipment. <i>Notify the student and suggest troubleshooting steps (if the student has the necessary permissions).</i> 4a. System fails to send the command to the equipment (if the student attempts and has control permissions). <i>Notify the student and suggest to retry (if the student has the necessary permissions).</i>

2 Hardware kit

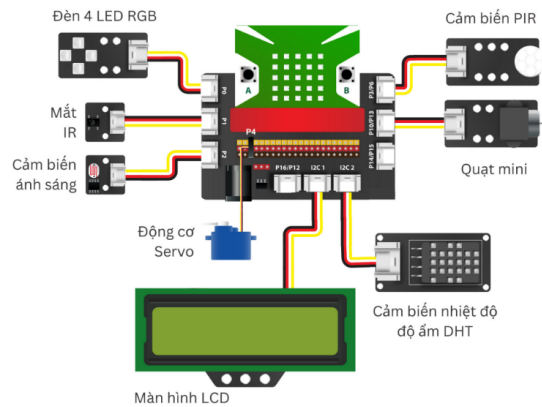


Figure 1: *IoT kit*

1. **Remote controller:** Adjust the device in case the system error or for testing.
2. **Circuit board:** Backbone circuit for Yolo:Bit serves as a source circuit.
3. **Yolo:Bit:** Programmable micro-processor that supports built-in functionalities for deploying ready-to-use IoT services.
 - Support USB-C, wifi and bluetooth connection for peripheral device. This help our system easily communicate with IoT server and its gateway.
 - Include of 2 button output for optimizing some simple operation.
 - Integrated mini speaker, led matrix (5x5) and useful Output modules for programming.
 - Support 21 extended pin.
4. **Servo engine:** Mini motor works under low-voltage environment.
5. **Relay:** Electrically operated switch.
6. **Infrared sensor:** Measuring infrared lights. Output a voltage in form of serialized integer.
7. **4 LED RGB** Electrically controllable RGB light.
8. **LCD** 16×2 Mini screen supporting various display modes.
9. **Distance sensor:** Measuring distance. Output a voltage in form of serialized integer.
10. **Light sensor:** Measure light intensity in lux. Output a voltage in form of serialized integer between 0 and 1024 representing light intensity in lux.
11. **MiniFan device:** A motor-like with built-in fan-blade, operates under low-voltage environment.

3 Implementation

3.1 Database design

3.1.1 Database selection

With `Users`, `LoginLog`, `Devices_Sensors`, `IssueLog` will be stored in our cloud database - MongoDB Atlas. We choose this database because we recognise there are not many tight relationship. Further more, the schema for the database is flexible and can be modified to satisfy the future requirements without much effort. In addition, there is strong support from our framework (NodeJS, ExpressJS) for MongoDB.

Besides, `Devices_SensorsLog` will be stored in cloud of the Adafruit server with the streaming data from MQTT protocol of Adafruit. We can get the history of information about devices and sensors through provided APIs and fetch data for our own website's purposes

3.1.2 Physical design

We choose a Document-oriented database, MongoDB, to implement our data model on the server. Document-oriented database is NoSQL, supports two most important features for our project: (1) horizontal scalability and (2) flexible schema. Uncertainty in the requirements of our project making schema evolution a necessity, therefore a NoSQL, Document-oriented database is the best fit.

Each table in Figure 9 is a root collection in our data model. MongoDB does not have sub-collections therefore modeling as collection hierarchy is not possible in this DBMS.

We deploy the database using MongoDB Atlas, software as a service provided as a cloud solution. With the capabilities of Atlas, setting up and running a MongoDB database becomes a hassle-free process, achievable with just a few clicks and within a matter of minutes.

Our data is JSON-formatted, and stored as BSON (binary-encoded JSON) as defined by MongoDB. JSON, which stands for JavaScript Object Notation, is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate. It is primarily used to transmit data between a server and a web application. JSON is text-based, making it language-independent, and it consists of key-value pairs arranged in a simple and readable structure.

Schemas in MongoDB is flexible and does not need to be homogeneous among documents in the same collections, allowing our database to ignore a field when its value is not existed. The collection schemas defined below describe all possible fields if their values exist, otherwise the field will be omitted.

Users Collection

```
1 {  
2   'userId': {type: String},  
3   'fullname': {type: String},  
4   'email': {type: String},  
5   'password': {type: String},  
6   'role': {type: JSON_Object},  
7   'phoneNum': {type: string},  
8   'schedule': {type: JSON_Object},  
9 }
```

The **Users** Collection stores information to identify an end user (Student/Teacher/Administrator) of the system.

- The **role** field defines the list permissions associated with their resources on the server.
- The **schedule** field defines allowed time for user to use a specific service of the system.

Example of **schedule** field:

```
1 [  
2   B1-211, Mon, from 7:00:00 to 10:00:00 '  
3 ]
```

LoginLog Collection

```
1 {  
2   'loginAt': {type: Datetime},  
3   'userID': {type: String},  
4   'sucessFlag': {type: Boolean},  
5   'ipAddress': {type: String}  
6 }
```

The **LoginLog** Collection stores system logs upon user login/logout.

- **userID** values are consistent with ones in **Users**.
- **sucessFlag** is to track status of attempts to evaluate system failure or exploitation.
- **ipAddress** is to track where is the device responsible for the login action.

Devices_Sensors Collection

```
1 {  
2   'gadgetID': {type: String},  
3   'type': {type: String},  
4   'name': {type: String},
```

```
5   'location': {type: String},
6   'curValue': {
7       lastValue: {type: String},
8       updatedAt: {type: Date}
9   }
10  'schedule': {type: JSON_Object}
11 }
```

The `Devices_Sensors` Collection stores output devices' information.

- `curValue` values include the last value and corresponding time for that modification to visualize on our UI.

IssueLog Collection

```
1 {
2   'logID': {type: String},
3   'severity': {type: String},
4   'type': {type: String},
5   'deviceID': {type: String},
6   'location': {type: String},
7   'feedback': {type: String},
8   'isFixed': : {type: Boolean},
9   'createdAt': {type: Date},
10 }
```

The `IssueLog` Collection stores the reported issue and its status if it is fixed or not.

- `feedback` contains the detail about the issue from users.
- `isFixed` is to track whether the issue is troubleshooted or not, default value is `False`.
- `createdAt` expresses the time that issue report sent.

Data constraints

1. The `gadgetID` must start with the first 3 letters of its type name and follow by 7 numbers, or more specific, in the format of `<abcXXXXXXX>`. For example, the ID of the first light in the class will be `LIG0000001`, or the ID of the 9th ceiling fan will be `FAN0000009`. It will be the same with `sensorId` and `deviceID`.
2. The `room` location of Device and Sensor will be limited to the certain available room in our university.
3. If there is new appended data in `ConfigLog`, `Device_SensorLog`, it will trigger the `Config`, `Devices_Sensors` table to update its current value.

3.2 Deployment diagram

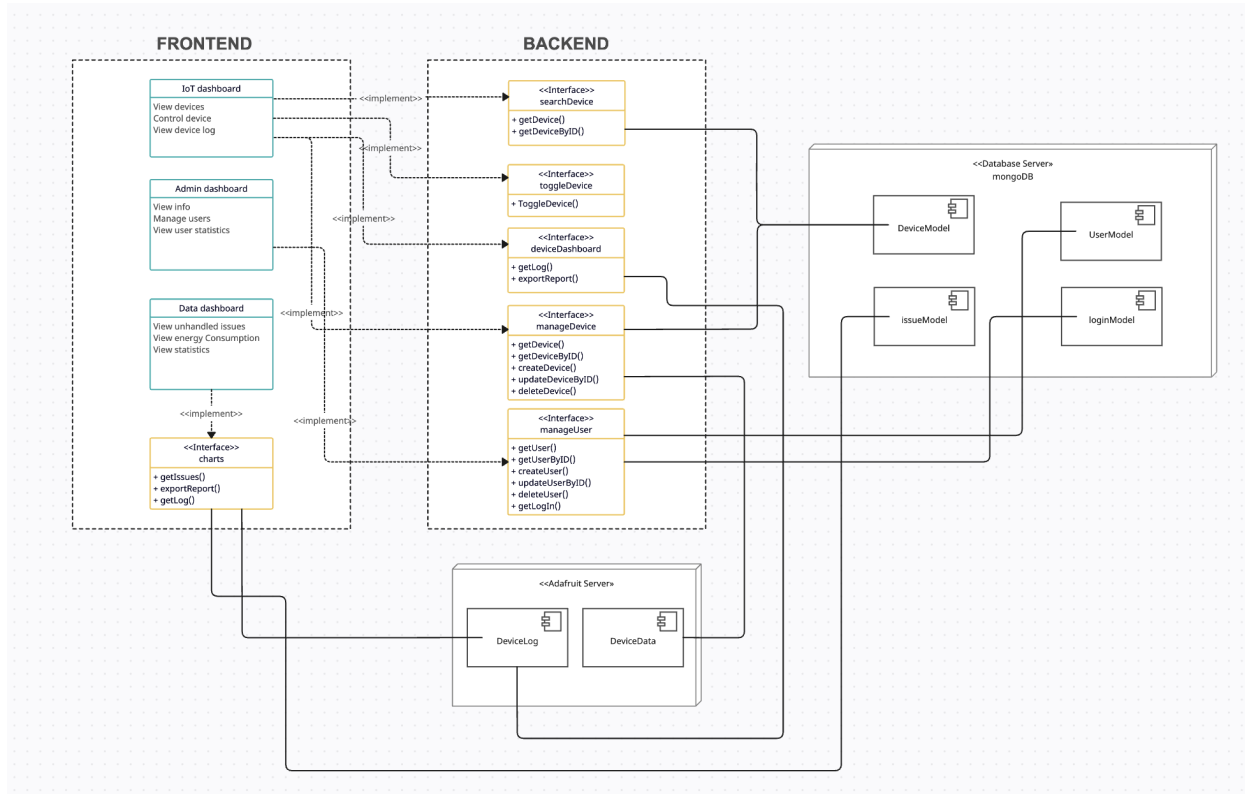


Figure 2: Deployment diagram

3.3 IoT Integration Code

For sensors and devices to function as required, we need to design and build embedded code. Below is the process of building code for devices to operate and communicate with Adafruit.

3.3.1 Initial setup

In this section, we have set up to connect the IoT system to the existing Adafruit server and provide the key to be able to write and read the feed. We've also updated the time, prepare function, state variables, and display a message once preparation is complete.

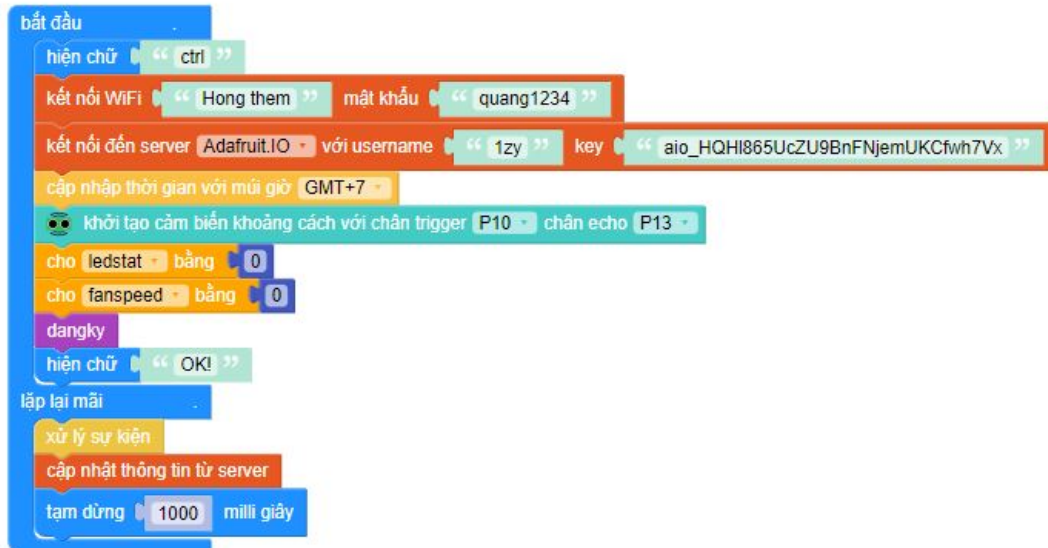


Figure 3: *Initial setup*

In an endless loop, we continuously update information and events.



Figure 4: *Every thing in this will loop forever*

3.3.2 Registration function

This function is largely responsible for controlling devices with information sent back when feeds in Adafruit are recorded. by checking the information received continuously from the command to update information from the server in a forever loop. The function will check the received value and update the status of the corresponding devices associated with the feeds. In this function, we have implemented control of lights, fans, and doors.

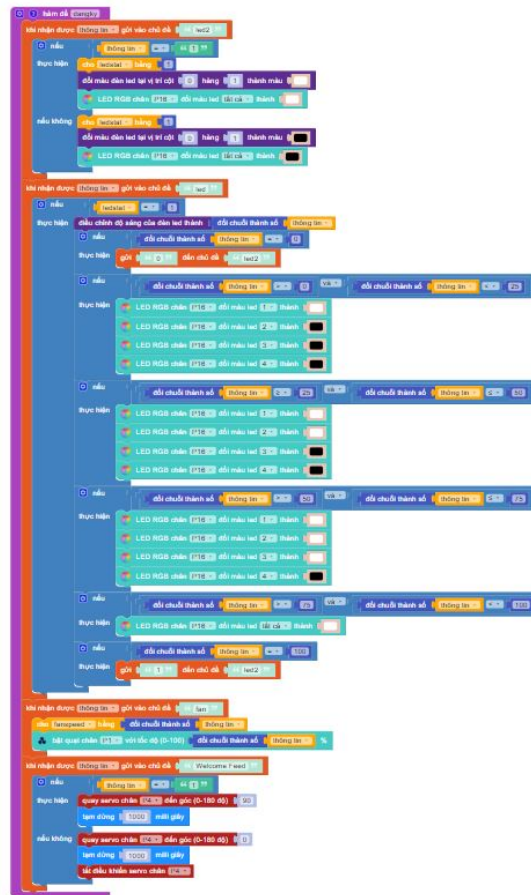


Figure 5: *Registration function*

3.3.3 Events

Events need to be executed after a certain period of time, depending on their nature, priority, and the amount of data points allowed to be sent to Adafruit. Instead of being processed from the top down, event handling code placed in a forever loop will work in such a way that on each event, the work in the event block will be processed and so on... These event blocks will take care of the following works.

- **Updating data:** this block is responsible for reading, displaying and sending data from sensors to LCD screen and feeds on Adafruit.
- **Handle cases exceeding the threshold:** We use event blocks to continuously check data from sensors for environmental conditions in the room and make adjustments from devices such as turning on fans and lights to ensure appropriate classroom conditions.
- **Automation:** We also use event blocks to implement automatic door functionality.

- **Notification by light:** Using LED lights to notify the status of the device as well as the environment.

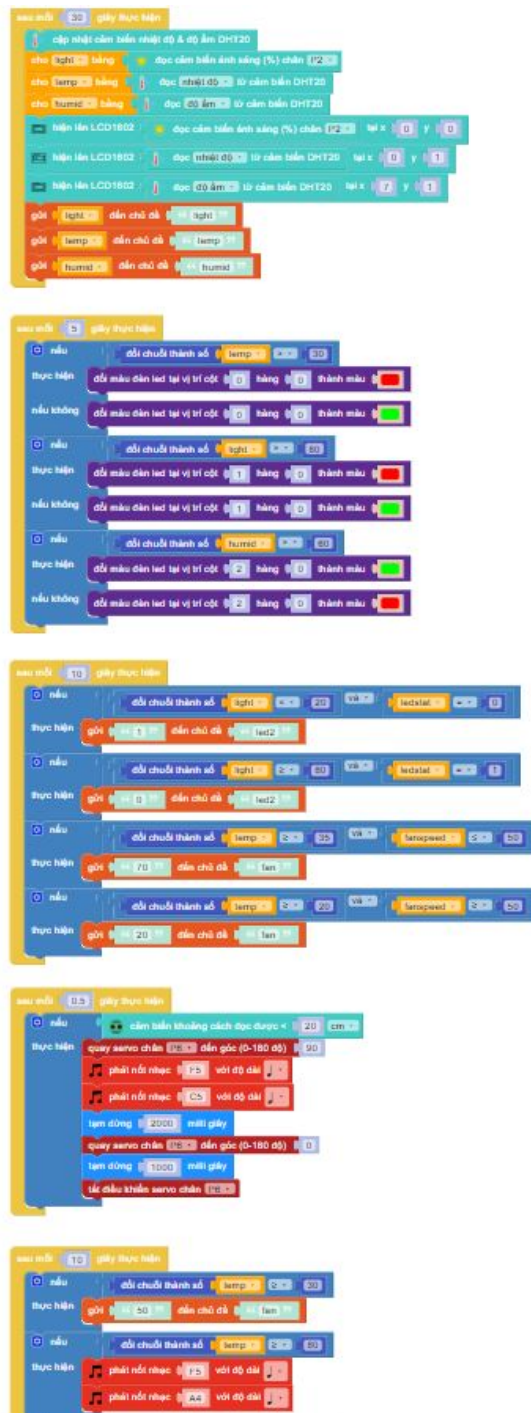


Figure 6: Event blocks

3.3.4 Full IoT system

After combining it all, we have an IoT system that has the following functions:

- Display and send information from sensors to users and Adafruit servers.
- Displays environmental status with LED lights.
- Check and adjust the device according to the value received from the corresponding feeds.
- Check and adjust the device when the value received from the sensors exceeds the threshold.
- Automation.

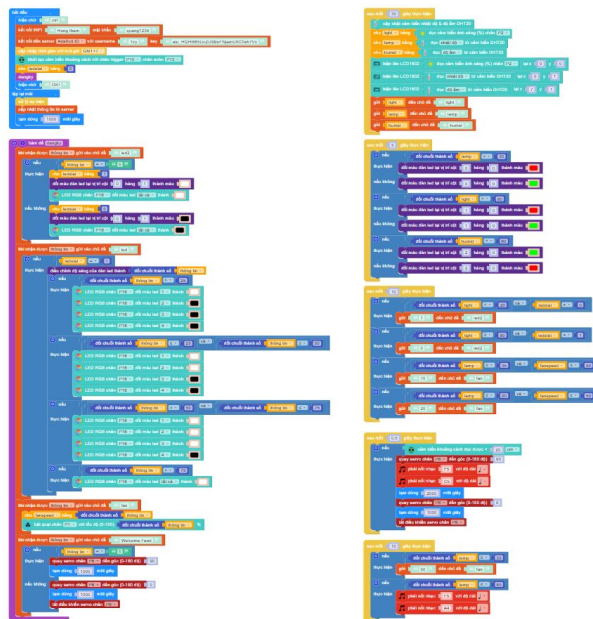


Figure 7: Full IoT system

3.4 Finished Product

3.4.1 Source control

Effective source control is crucial for the collaborative and error-free development of our project. It ensures that changes by different team members are tracked, and the project history is preserved for review and rollback if necessary. Our project's source code is hosted on GitHub, which can be accessed at this [link](#). All development is primarily conducted on the **newer-branch**, allowing us to leverage the benefits of distributed revision control and branch-specific workflows.

3.4.1.1 Distributed revision control system

We use Git as our distributed version control system, which allows multiple developers to work on a particular project without the need to share a common network. Git enables robust branching and merging, providing a reliable backup and undo system, and supports non-linear development through its powerful branch handling capabilities.

3.4.1.2 Source code structure

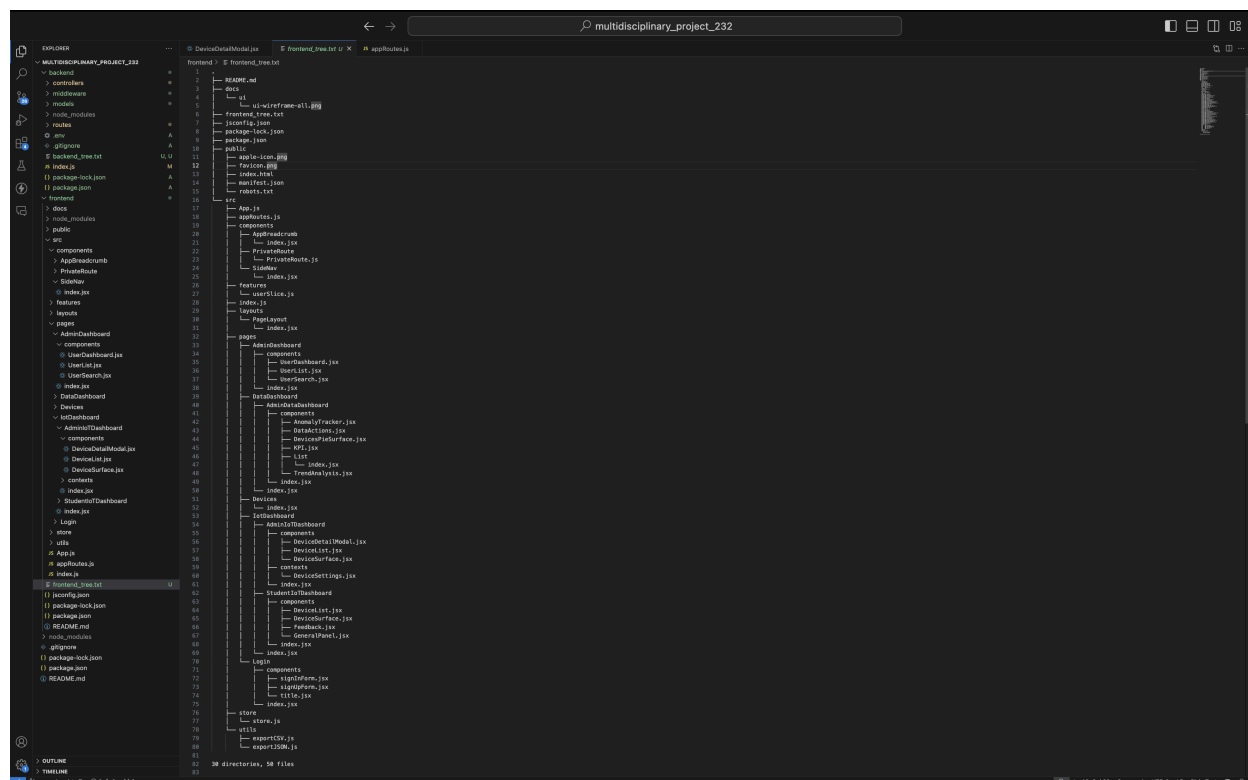


Figure 8: *Frontend Structure*

1. Frontend:

The frontend of our application is structured to maximize usability and performance. It is built using React.js, which enables modular component management and dynamic user interface updates. The structure is designed to be scalable and maintainable, facilitating the development and integration of new features as the application evolves.

- **Components:** Organized into small, reusable components, each serving a specific function within the application.
 - `AppBreadcrumb` - Provides navigational breadcrumbs to enhance user experience.
 - `PrivateRoute` - Manages route access based on user authentication status.
 - `SideNav` - Implements the sidebar navigation component.
- **Pages:** Represents the views that are rendered based on different routes.
 - `AdminDashboard` - Central hub for admin users, including user management and search functionalities.
 - `DataDashboard` - Displays data analytics and tracking, with components for data actions, trend analysis, and KPI visualization.
 - `Devices` - Manages and displays information related to IoT devices connected to the system.
 - `IotDashboard` - Dedicated dashboards for different user roles (Admin, Student) to interact with IoT device settings and data.
 - `Login` - Handles user authentication, including sign-in and sign-up forms.
- **Layouts:**
 - `PageLayout` - Defines the general layout of pages, including headers, footers, and sidebar navigation.
- **Features:**
 - `userSlice.js` - Manages the state related to user data using Redux Toolkit.
- **Store:**
 - `store.js` - Centralizes the application state management using Redux, facilitating state sharing across components.
- **Utils:**
 - `exportCSV.js` and `exportJSON.js` - Provide utilities for exporting data in CSV and JSON formats, supporting data usability and portability.

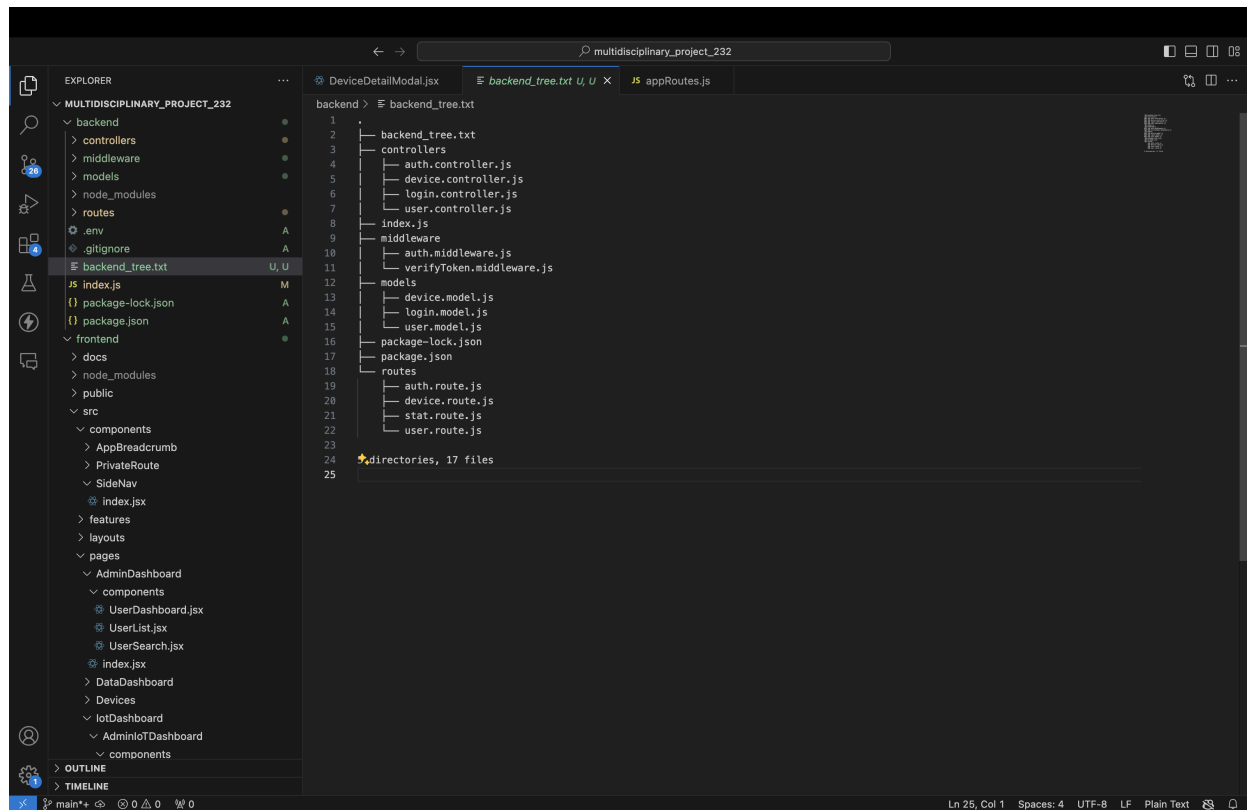


Figure 9: Backend Structure

2. Backend:

The backend architecture of our application is designed for high availability and scalable performance. It is primarily developed using Node.js and Express, which supports our application's requirements for real-time data processing and integration with various databases and IoT devices. Our backend is structured to facilitate clear separation of concerns and maintainability through a well-organized directory and file structure:

- **Controllers:** Handles business logic and interactions between the models and the views.
 - `auth.controller.js` - Manages authentication logic.
 - `device.controller.js` - Controls interactions related to device data.
 - `login.controller.js` - Processes login requests.
 - `user.controller.js` - Handles user information management.
- **Middleware:** Intercepts and processes requests before they reach the controllers.
 - `auth.middleware.js` - Manages authentication processes.
 - `verifyToken.middleware.js` - Verifies tokens for secured routes.

- **Models:** Defines data structures and models.
 - `device.model.js` - Defines the structure for device data.
 - `login.model.js` - Represents the login data model.
 - `user.model.js` - Describes the user data model.
- **Routes:** Defines the API routes and associates them with specific controller functions.
 - `auth.route.js` - Routes for authentication processes.
 - `device.route.js` - API routes for device interactions.
 - `stat.route.js` - Routes for statistics and analytics.
 - `user.route.js` - Manages routes related to user information.

Each component is modular and designed to interact seamlessly, ensuring that enhancements and maintenance can be managed efficiently. The use of Node.js and Express not only facilitates rapid development but also ensures that the backend is capable of handling asynchronous operations and extensive data exchange effectively, critical for real-time applications.

3.4.2 Final product

Expanding on the system designs outlined earlier, we have successfully developed a website that fully meet our system requirements. These pages have been crafted with the end user in mind, featuring an intuitive and user-friendly interface that enables users to navigate and interact with the system effortlessly.

Our design aims to provide an intuitive, navigable User Interface that fosters the best experiences for users on our platform. **YoloHome: SMART CLASSROOM** is constructed with human-centric in mind, in acknowledgement that students and teachers may come from diverse technical background and capabilities. In order to achieve the above objectives, we employ **three principles** that would guide us throughout the Front-end development of the product:

1. Do not re-invent the wheel. Use prebuilt, production-ready, familiar components^[?].
2. Use default color themes, with the predefined color wheel **that invokes emotions**^[?].
3. Adheres to existing conventions and design guidelines^[?].

Our wireframe mockups are available at **Github**. We implement our User Interface using **Joy UI**, a production-ready, open source **React** component library that implements **MUI** design principles.

3.4.2.1 Portal

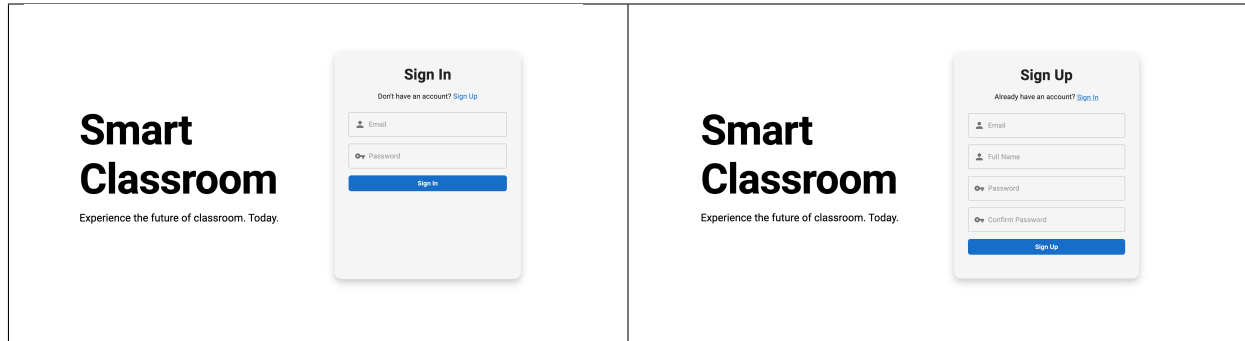


Figure 10: Portal page (Sign In/Sign up)

Portal screen is the landing page which provides Sign In and Sign Up services, via password-based authentication.

The page is divided into 2 columns: the left is the first impression of our product clarifying what our product is and the delivered value; the right is the input panel in which the user enters credentials to proceed. If the credential is not valid, users are not able to proceed to other pages. The experience that we offer is simple, intuitive, and familiar to most portal pages nowadays.

3.4.2.2 Data Dashboard

As an administrator, after successful login, the user will be directed to one of 3 main pages which is data dashboard. There are 2 columns that can be clearly observed.

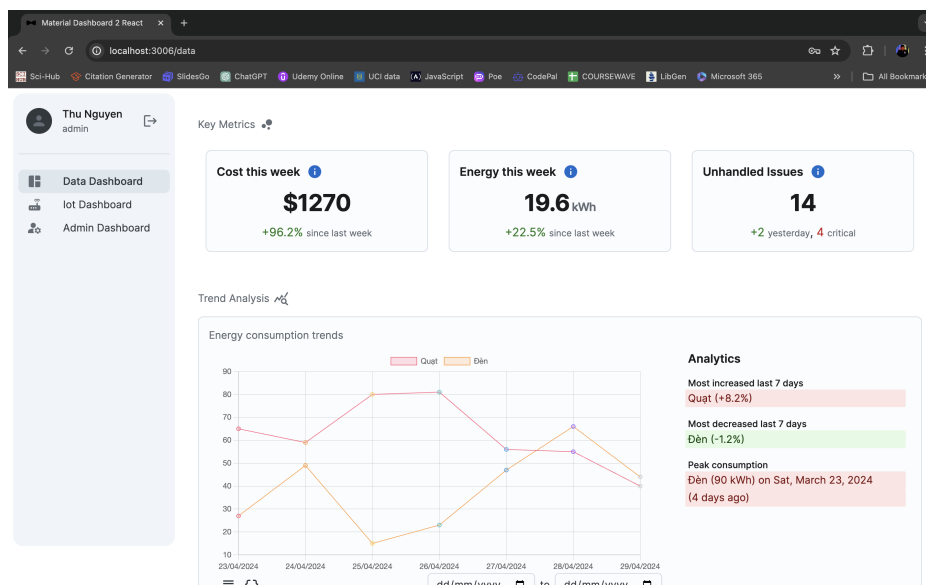


Figure 11: Administrator's Data Dashboard page

The first column on the left hand side expresses who user is now logged in with their own role below username, followed by a list of 3 pages with the emphasis for the current page.

The second column occupies the majority size of the page which also visualizes the data of our IoT system. The head line consists of 3 figures which are the cost for the electricity bill this week, energy consumption and incurring problems for the whole system, respectively. The comparison between the current index and last week's index is shown below the main figure for cost and energy consumption, and it is a detailed issue number with the right-most box.

The main chart is a line graph that demonstrates the tendency of energy consumption by day, presented by 2 distinct lines: one for Fan and one for Light. The time span filter is also provided in the bottom right corner of the graph. An analytic table can be transparently seen on the right side, consisting of the most highlighted changes and the peaking figures.

As a student, this dashboard page is totally the same except the tiny difference about the role of the user.

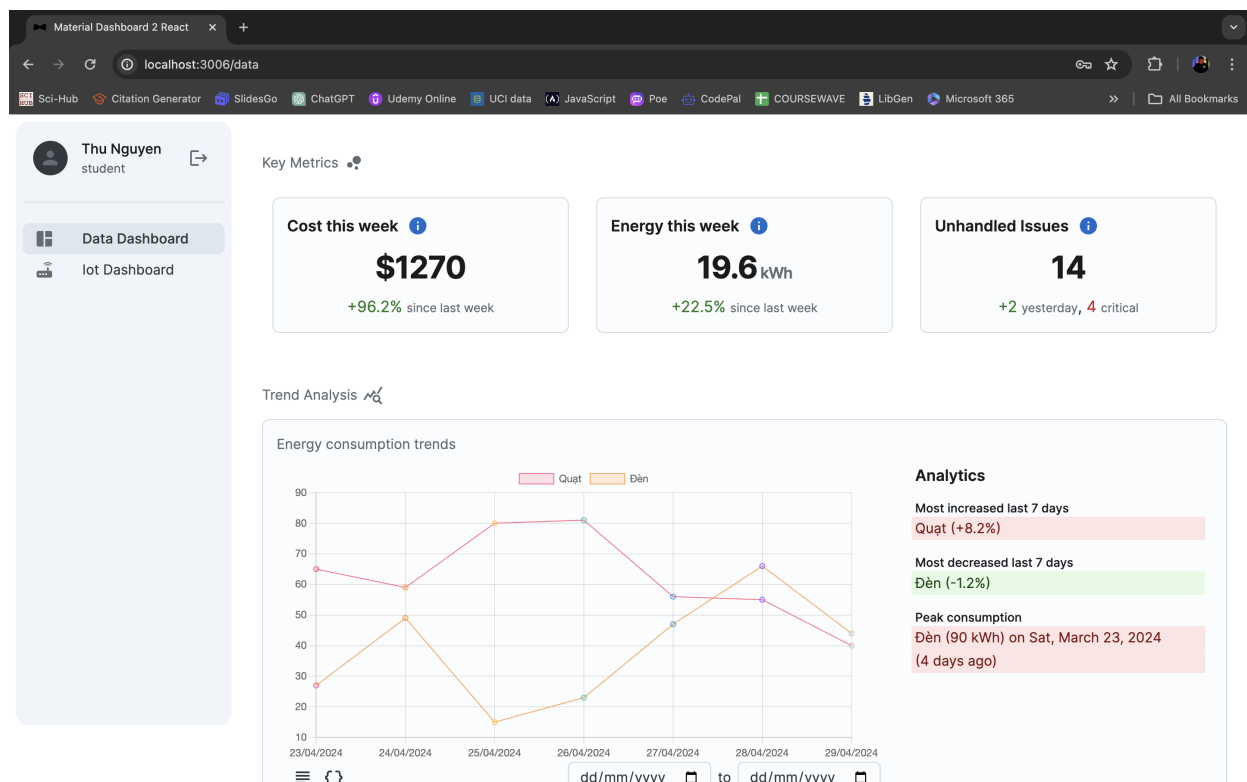


Figure 12: User's Data Dashboard page

With dashboards, users can observe the fluctuations mainly in the energy expenditure. This can help the administrator to know about the status of the IoT system and give some decisions to improve the bad scenarios. Moreover, this also can improve the awareness in saving electricity if

the user is a student.

3.4.2.3 Administrator's User Dashboard

Besides the management in IoT system energy consumption, the user management is one of the concerns of the admin.

There is a search bar on the top of the table used for filtering users by their name. Whereas, the main occupation is used for a table containing main information of users: user's name, corresponding role and email. Moreover, there are several setting operations that can be applied on each user, including modifying roles and removing users with 2 icons on the same line. With the long list of users we can use the pagination on the bottom right corner to move between 9 line sub-lists.

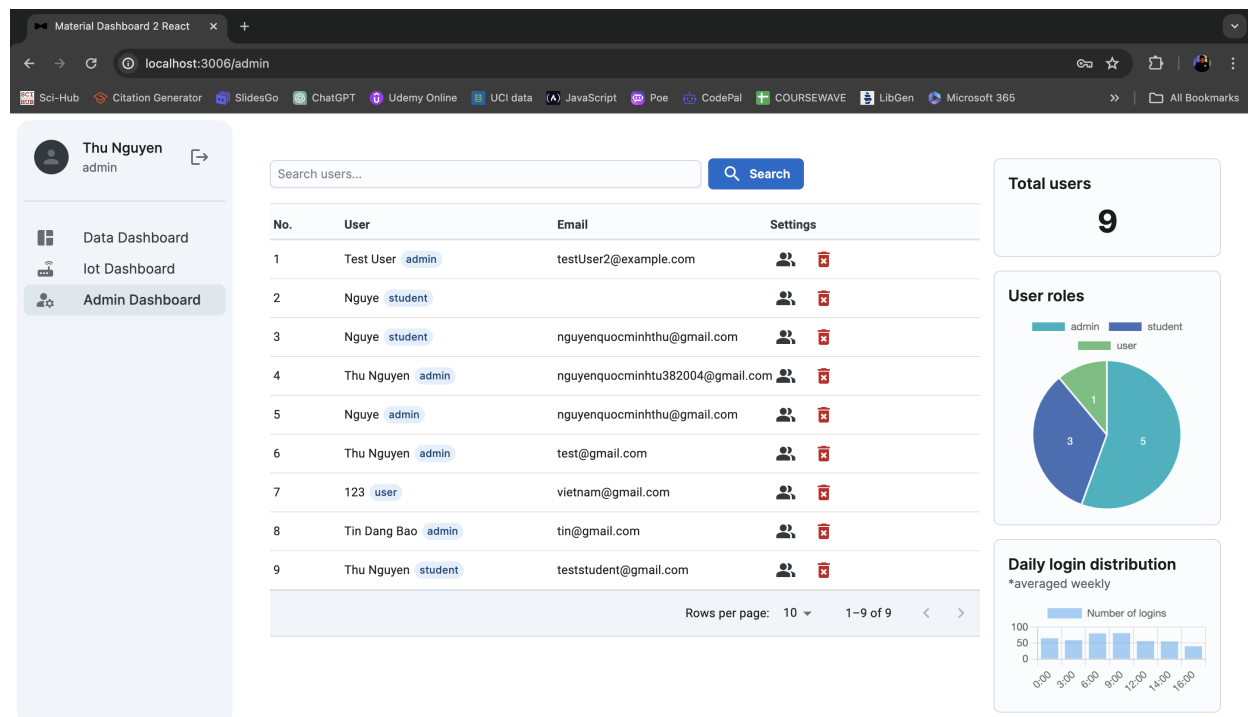


Figure 13: Admin's User Dashboard page

On the right side, we can see an analytic table which is divided into 3 parts: total users, user roles' proportions and finally the daily login distribution. This combination supports admin to have the general size of clients participating in this system with the occupation for each role in addition with login frequency for the whole system by date.

Getting deeper into 2 functionalities: role modification and user removal, are also provided in

our website. After any changes, the administrator must confirm that action to complete successfully.

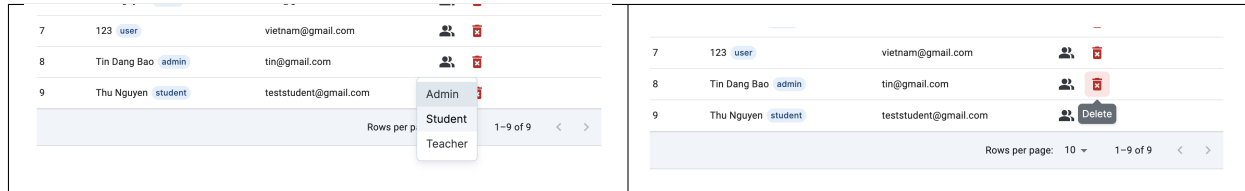


Figure 14: Modification and Removal option

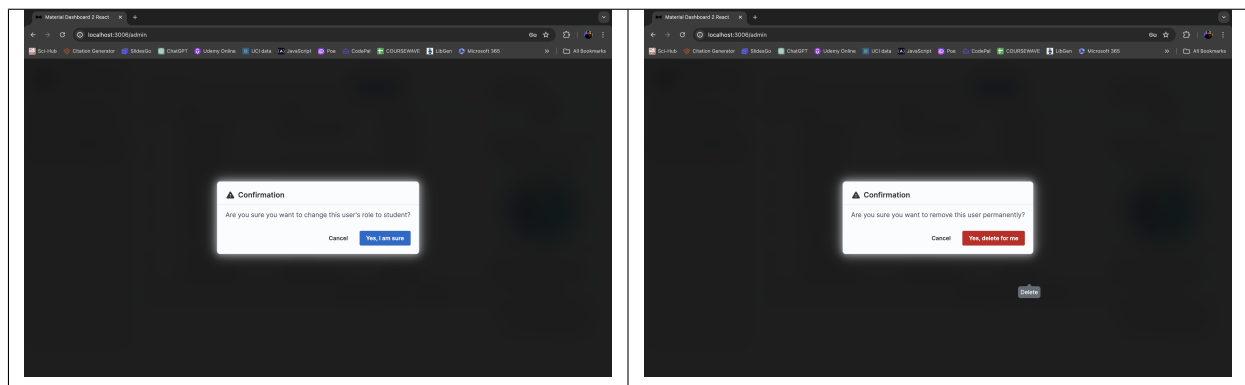


Figure 15: Modification and Removal confirmation

3.4.2.4 IoT Dashboard

IoT Dashboard is the central page for viewing and interacting with data streamed from the devices, as well as providing real-time controls to the devices. As the product is the system with IoT devices, this page is selected to be the Home page. User would see this screen after the Login process.

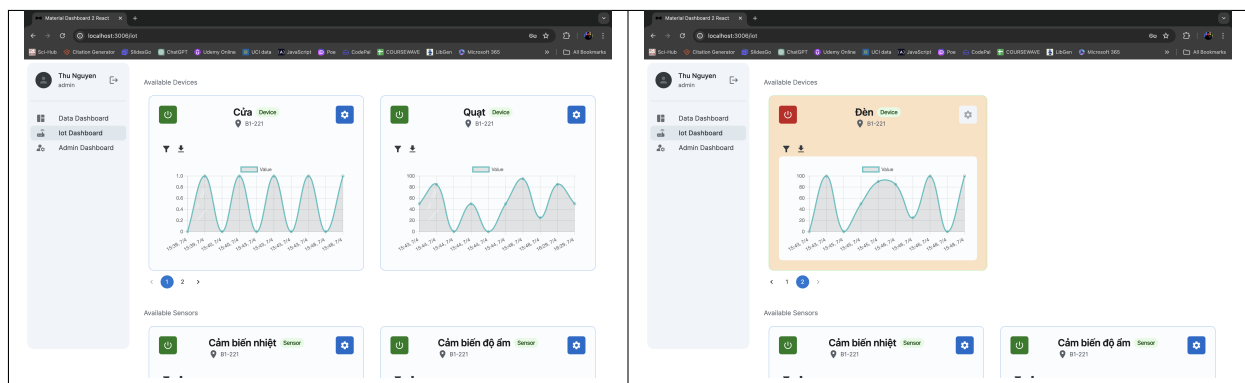


Figure 16: Admin's IoT Dashboard page - Devices

In our system, 3 devices are used for demonstration: Fan, Light and Door. Each device is

shown with its name and its location, or classroom for specific, and being attached with its status line chart. We use the horizontal pagination to move between devices. Each device has an on/off button on top left, and the setting button on the top right, whose detail will be presented later. The graph of each device illustrates the data for a certain time duration which can be modified by the administrator by the clicking on time filtering icon. While the icon nearby is used for downloading the data in several formats, such as JSON and csv.

When clicking on the on/off button, the corresponding device will follow the control real-time and the changes can be seen on the graph. With the setting button, we can see 2 functionalities. The first one is setting specific values for Light and Fan, and this brightness or speed will be expressed on the device after clicking the Apply button. In addition, setting a threshold for the individual device can help the admin to keep the device in a safe status preventing any abnormal controlling action by setting back to the standard value.

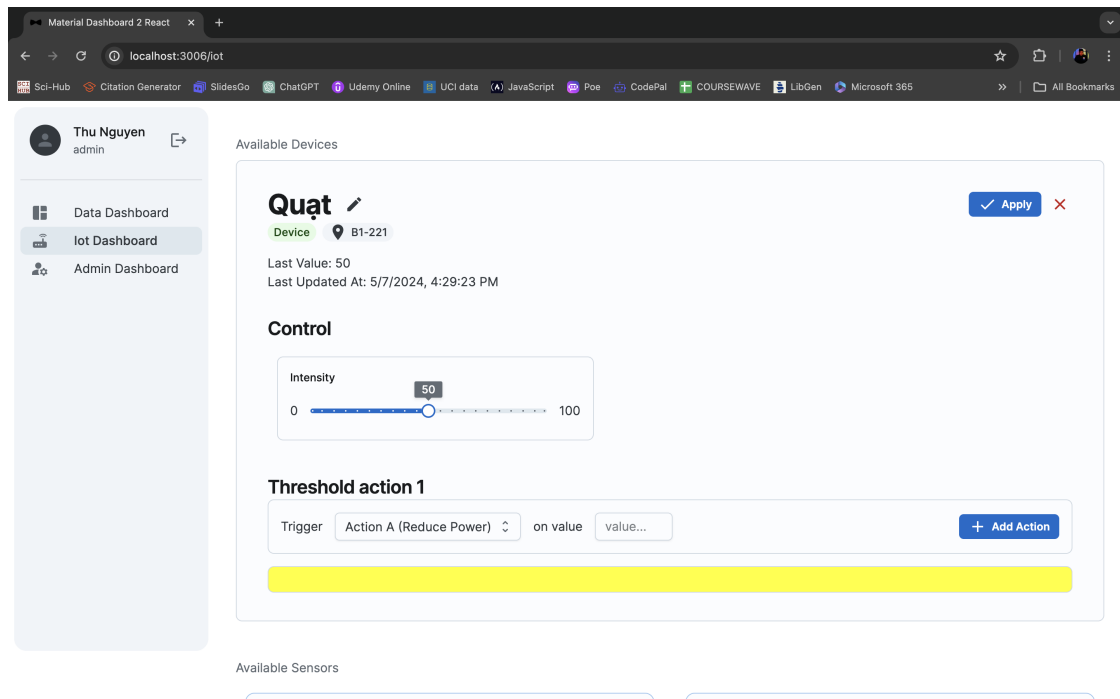


Figure 17: *Admin's IoT Dashboard page - Device Setting*

Besides the device controllers, we can also observe the trend in status of the sensor real-time, parallel with switching on and off the sensor similar with the devices' functionalities.

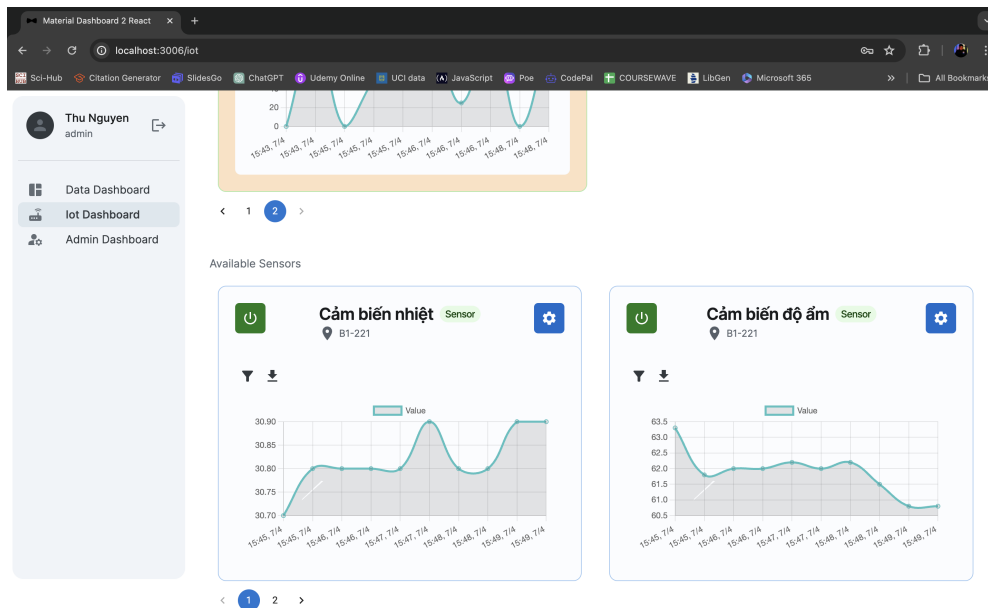


Figure 18: Admin's IoT Dashboard page - Sensors

When using website in a normal user role, there will be some limitation on functionalities, and need to follow their schedule to have permission to control device. In details, student or normal user can see their working schedule at a certain room in university and can observe the device list in that room at the right time. Instead having full permission operating on the devices and sensors, users only can switch them on or off.

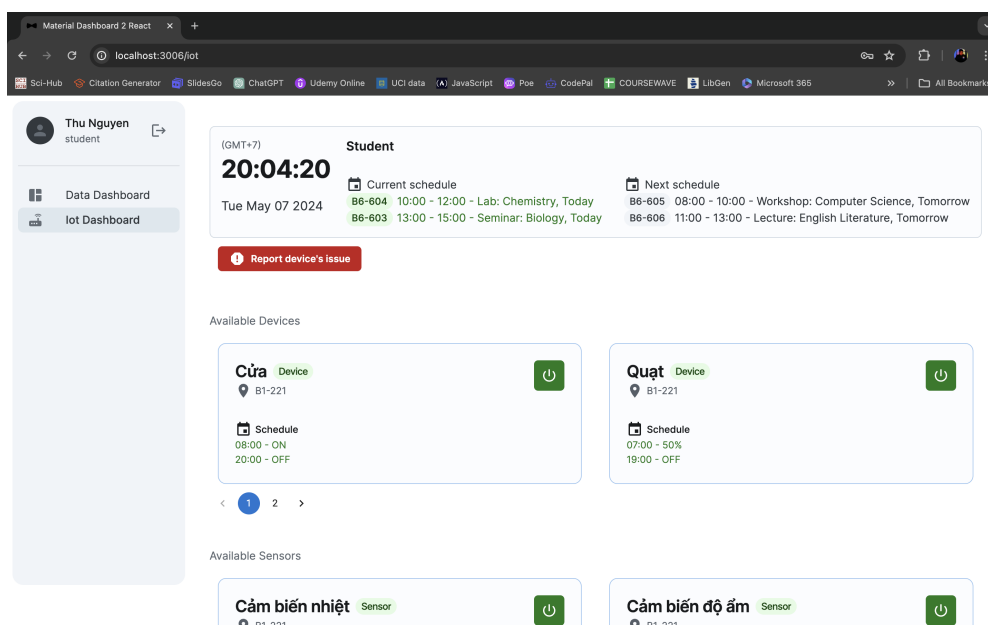


Figure 19: User's IoT Dashboard page

However, there is an issue report button provided for user to warn the administrator some problems that happen on the system for a timely troubleshooting. User needs to fill in 3 fields for a warning report: emergence level, device name, and an extra text field; finally clicking submit to send report.

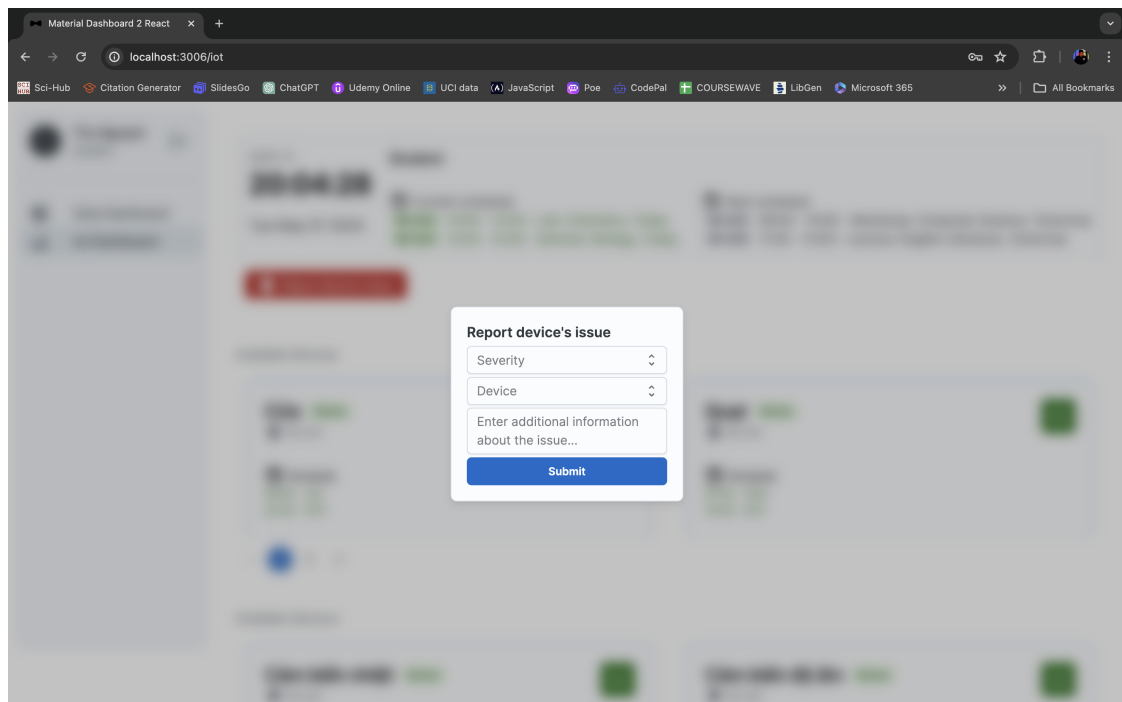


Figure 20: *User's IoT Dashboard page - Issue report*

3.5 Demonstration

You can watch our final demonstration for the YOLO IoT Classroom System via this [link](#).

4 Project Evaluation

4.1 Summary of the project's results

The project successfully implemented a robust system capable of managing user authentication and interactive dashboards for real-time IoT device monitoring and data visualization. Essential functionalities like system login/logout, real-time data interaction via IoT dashboards, and flexible data representation and exporting have been fully developed and integrated.

Throughout this **multidisciplinary project**, our team has gained significant insights into the integration of software and hardware components, crucial for the development of IoT-based systems. Key learnings include:

- **Cross-disciplinary Collaboration:** One of the primary lessons learned was the importance of effective communication and collaboration between the software (CS) and hardware (CE) teams. Ensuring that both teams were aligned on project goals and technical requirements was essential for seamless integration and functionality of the IoT devices.
- **IoT Device Integration:** We learned about various challenges associated with integrating IoT devices, including issues related to device compatibility, network reliability, and data synchronization. Through practical application, we developed strategies to address these challenges, such as adopting universal communication protocols and performing rigorous testing scenarios.
- **Real-time Data Handling:** The project provided practical experience in managing and processing real-time data from IoT devices. This included learning how to efficiently collect, process, and display data in a user-friendly manner on our dashboards, which was crucial for providing actionable insights to users.
- **Information Management:** The project sharpened our skills in managing complex information flows within IoT systems. We developed an understanding of how to structure databases effectively. This was critical for supporting the functionality of our IoT devices and ensuring that all system components could access up-to-date and accurate information.
- **User Data Handling:** We focused on the importance of responsibly managing user data, especially in scenarios involving multiple user roles and permissions. Learning to implement fine-grained access controls and auditing mechanisms was essential for ensuring that user data was not only protected but also correctly utilized according to user roles and permissions.
- **UI/UX Skills:** Through the design and implementation of the project's user interfaces, we significantly enhanced our UI/UX design skills. The process taught us about the importance of creating intuitive and user-friendly designs that accommodate users with varying levels of technical expertise.

4.2 Achievements

- **User Authentication:** We have implemented a robust, secure login and logout system tailored for various user roles including administrators, students, and teachers. This system incorporates advanced security measures like two-factor authentication and encrypted data storage to ensure both the security and usability of the platform. The flexibility of this system allows for easy onboarding of new users and roles as the system scales.
- **IoT Dashboard Functionality:** The IoT dashboard is fully operational, providing real-time interactive access to connected devices and sensors. Users can not only view real-time data but also adjust device parameters such as temperature settings, switch states, and other

operational parameters. This dashboard enhances user engagement by providing a responsive and intuitive interface for device management, complete with real-time status updates and automated control loops.

- **Data Dashboard Enhancements:** Our data dashboard now supports multiple data visualization formats and can export data in several file types including PDF, Excel (XLSX), CSV, JSON, and XML. This versatility supports a wide range of data analysis and sharing needs, enabling users to choose the best way to present and analyze data according to their specific requirements. The dashboard includes interactive charts, graphs, and real-time data feeds, making it a powerful tool for data-driven decision making.
- **Threshold Trigger Mechanism:** We developed and integrated an advanced threshold trigger mechanism into our system that automatically initiates actions on connected IoT devices when certain predefined conditions are met. This feature is particularly useful for maintaining optimal environmental conditions or for alerting users to potential issues before they become critical. It uses complex algorithms to process incoming data and determine when to execute predefined actions, greatly enhancing the system's automation and responsiveness.

4.3 Challenges

- **Integration Complexity:** Integrating diverse IoT devices and ensuring consistent data handling and device control was challenging, requiring extensive testing and refinement.
- **Security Concerns:** Ensuring robust security measures for user authentication and data protection while maintaining system performance posed significant challenges.
- **User Interface Usability:** Designing an intuitive user interface that accommodates varied user expertise levels required iterative design and user feedback integration.

4.4 Further Improvement

- **AI Integration for Automation:** In future developments, we plan to integrate AI-driven functionalities to automate system operations such as adjusting brightness, temperature, and device control based on environmental factors and user preferences.
- **AI Authentication and Authorization:** Enhancing security and user authentication with AI-assisted recognition and permission systems will be prioritized to provide a more secure and personalized user experience.
- **Extended IoT and Data Dashboard Features:** Continuous improvement of the IoT and data dashboards to include more interactive features and data analytics tools will be pursued to enhance user engagement and system utility.



References

- [1] Bottomley, Paul. (2006), The interactive effects of colors and products on perceptions of brand logo appropriateness. In: *Marketing Theory*, 6th ed, S. 63–83.
- [2] Jakob Nielsen. (2000), *Jakob's Law of the Internet User Experience - UX Tigers*, 66(3), 1116-1135.
- [3] Interaction Design Foundation. (2024), *Design Guidelines*, <https://www.interaction-design.org/literature/topics/design-guidelines>.
- [4] Juice, Inc. (2009), *Guide to Dashboard Design*.